



Proyecto 1 – Agenda Digital Tres Patitos

CURSO Bases de Datos GR

II semestre 2026

Alejandro Zuñiga Martínez

Carné: 2025113729

Prof. Jacqueline Solis

Fecha de entrega: 15/09/2026

Contenido

Introducción 3

Matriz de trazabilidad y Estado de Implementación..... 3

Módulo de Gestión de Ubicaciones..... 3

Módulo de Disponibilidad de Usuarios y Gestión de Tiempos..... 3

Módulo de Tareas Asociadas a Eventos..... 4

Módulo de Gestión de Ubicaciones 5

Modelo Conceptual 5

Modelo Lógico..... 5

Módulo de Disponibilidad de Usuarios y Gestión de Tiempos 6

Modelo Conceptual 6

Modelo Lógico..... 6

Modelo Físico 6

Módulo de Tareas Asociadas a Eventos 7

Modelo Conceptual 7

Modelo Lógico..... 7

Modelo Físico 7

Evidencias Visuales 8

Pestaña de Ubicaciones 8

Pestaña de Tareas..... 8

Pestaña de Disponibilidad 9

Pestaña de Eventos con el selector de Ubicación..... 9

CRUD 10

CRUD de Modulo de Ubicaciones 10

CRUD de Módulo de Tareas..... 12

CRUD de Módulo de Disponibilidad..... 14

Validación de Datos 16

Repositorio en Git 17

Introducción

En este informe se presenta el trabajo realizado sobre el sistema "Agenda Digital Tres Patitos", una aplicación de agenda construida en PostgreSQL con interfaz en Python. La base de datos ya contaba con usuarios, eventos, categorías y participaciones, y a partir de ahí se decidió ampliar el sistema con tres módulos nuevos: Gestión de Ubicaciones, Tareas Asociadas a Eventos y Disponibilidad de Usuarios y Gestión de Tiempos. El módulo de Eventos Recurrentes no se incluyó en esta ampliación.

Para trabajar cada módulo se siguió el mismo camino: primero se diseñó el Modelo Conceptual con su diagrama, luego se pasó al Modelo Lógico definiendo las tablas, después se construyó el Modelo Físico ejecutando el SQL en PostgreSQL, y finalmente se integró todo a la Interfaz Gráfica de la aplicación. En las siguientes secciones se muestra la matriz de trazabilidad junto con el desarrollo completo de cada módulo.

Matriz de trazabilidad y Estado de Implementación

Módulo de Gestión de Ubicaciones

Req.	Fase	Tarea	Estado
RF-08	1. Modelo Conceptual	Definir la entidad ubicaciones y sus atributos obligatorios	Completado
	2. Modelo Lógico	Estructurar la tabla relacional ubicaciones	Completado
	3. Modelo Físico	Codificar el CREATE TABLE ubicaciones en PostgreSQL	Completado
	4. Interfaz Gráfica	Desarrollar pestaña y formulario de ubicaciones	Completado
RF-09	1. Modelo Conceptual	Establecer relación Eventos–Ubicaciones	Completado
	2. Modelo Lógico	Incorporar id_ubicacion como FK en eventos	Completado
	3. Modelo Físico	Ejecutar ALTER TABLE para la relación referencial	Completado
	4. Interfaz Gráfica	Selectores para consultar histórico por ubicación	Completado
RF-10	3. Modelo Físico	Consulta/vista de ranking de recintos	Completado
	4. Interfaz Gráfica	Panel de reportes de solo lectura	Completado

Módulo de Disponibilidad de Usuarios y Gestión de Tiempos

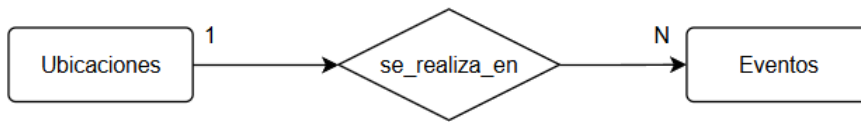
Req.	Fase	Tarea	Estado
RF-11	1. Modelo Conceptual	Definir disponibilidades y tipos disponibilidad	Completado
	2. Modelo Lógico	Tablas y FK hacia usuarios	Completado
	3. Modelo Físico	CREATE TABLE de tablas y catálogos	Completado
	4. Interfaz Gráfica	CRUD de disponibilidades	Completado
RF-12	3. Modelo Físico	Consulta de traslapes temporales	Completado
	4. Interfaz Gráfica	Integrar detección de usuarios libres en GUI	Completado

Módulo de Tareas Asociadas a Eventos

Req.	Fase	Tarea	Estado
RF-15	1. Modelo Conceptual	Definir la entidad tareas y sus atributos	Completado
	2. Modelo Lógico	FK hacia evento y usuario responsable	Completado
	3. Modelo Físico	CREATE TABLE tareas con estados predefinidos	Completado
	4. Interfaz Gráfica	Formularios CRUD de tareas	Completado
RF-16	3. Modelo Físico	Consultas de rendimiento y vencimientos	Completado
	4. Interfaz Gráfica	Desplegar métricas en pantalla	Completado
RF-17	3. Modelo Físico	Reporte de tareas activas y vencidas	Completado
	4. Interfaz Gráfica	Integrar reporte en el módulo de tareas	Completado

Módulo de Gestión de Ubicaciones

Modelo Conceptual



Modelo Lógico

UBICACIONES (id_ubicacion [PK], nombre, direccion, ciudad, capacidad)

EVENTOS (... , id_ubicacion [FK])

Modelo Físico

```
CREATE TABLE ubicaciones (  
    id_ubicacion SERIAL PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    direccion VARCHAR(150) NOT NULL,  
    ciudad VARCHAR(50) NOT NULL,  
    capacidad INT NOT NULL  
);
```

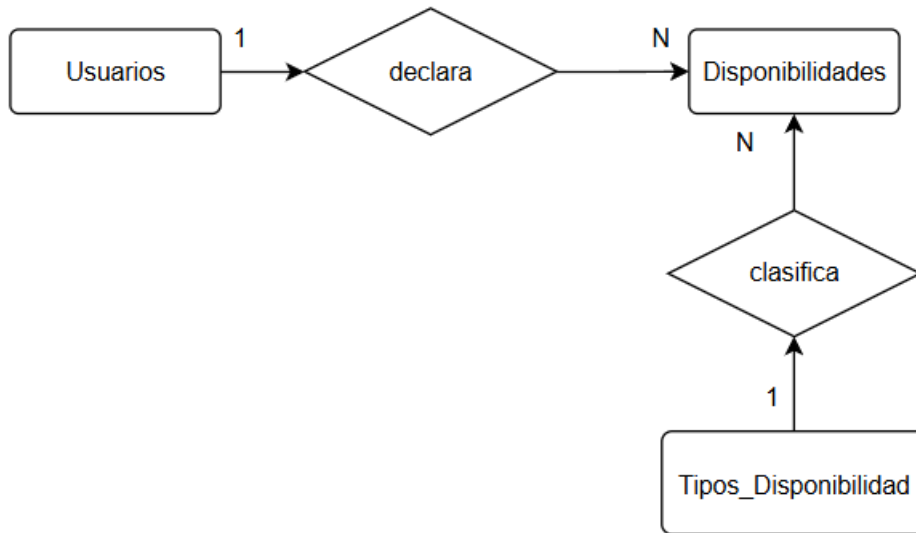
```
ALTER TABLE eventos  
ADD COLUMN id_ubicacion INT REFERENCES ubicaciones(id_ubicacion);
```

```
CREATE VIEW vista_ranking_ubicaciones AS  
SELECT  
    u.id_ubicacion, u.nombre, u.ciudad,  
    COUNT(e.id_evento) AS total_eventos  
FROM ubicaciones u  
LEFT JOIN eventos e ON e.id_ubicacion = u.id_ubicacion  
GROUP BY u.id_ubicacion, u.nombre, u.ciudad  
ORDER BY total_eventos DESC;
```

```
CREATE VIEW vista_traslapes_ubicaciones AS  
SELECT  
    e1.id_ubicacion,  
    ub.nombre AS ubicacion,  
    e1.id_evento AS evento_1,  
    e2.id_evento AS evento_2,  
    e1.titulo AS titulo_1,  
    e2.titulo AS titulo_2,  
    e1.fecha_inicio AS inicio_1,  
    e1.fecha_fin AS fin_1,  
    e2.fecha_inicio AS inicio_2,  
    e2.fecha_fin AS fin_2  
FROM eventos e1  
JOIN eventos e2  
    ON e1.id_ubicacion = e2.id_ubicacion  
    AND e1.id_evento < e2.id_evento  
    AND e1.fecha_inicio < e2.fecha_fin  
    AND e2.fecha_inicio < e1.fecha_fin  
JOIN ubicaciones ub ON ub.id_ubicacion = e1.id_ubicacion  
WHERE e1.id_ubicacion IS NOT NULL;
```

Módulo de Disponibilidad de Usuarios y Gestión de Tiempos

Modelo Conceptual



Modelo Lógico

TIPOS_DISPONIBILIDAD (id_tipo [PK], nombre)

DISPONIBILIDADES (id_disponibilidad [PK], fecha, hora_inicio, hora_fin, id_usuario [FK], id_tipo [FK])

Modelo Físico

```
CREATE TABLE tipos_disponibilidad (  
    id_tipo SERIAL PRIMARY KEY,  
    nombre VARCHAR(30) NOT NULL UNIQUE  
);
```

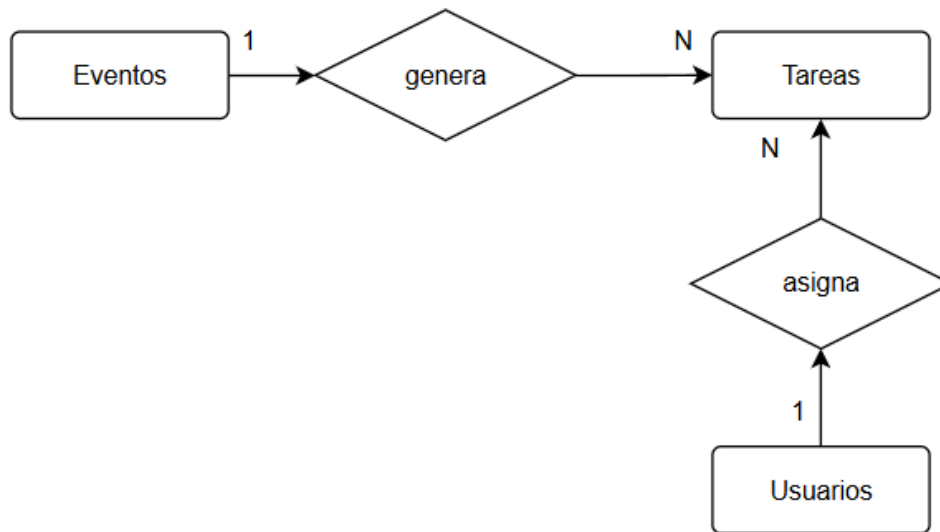
```
INSERT INTO tipos_disponibilidad (nombre) VALUES  
( 'Disponible'), ( 'Ocupado'), ( 'No disponible');
```

```
CREATE TABLE disponibilidades (  
    id_disponibilidad SERIAL PRIMARY KEY,  
    fecha DATE NOT NULL,  
    hora_inicio TIME NOT NULL,  
    hora_fin TIME NOT NULL,  
    id_usuario INT NOT NULL REFERENCES usuarios(id_usuario),  
    id_tipo INT NOT NULL REFERENCES tipos_disponibilidad(id_tipo),  
    CONSTRAINT check_horas_disp CHECK (hora_fin > hora_inicio)  
);
```

```
CREATE VIEW vista_traslapes_disponibilidad AS  
SELECT  
    d1.id_usuario, d1.id_disponibilidad AS disponibilidad_1, d2.id_disponibilidad AS disponibilidad_2,  
    d1.fecha, d1.hora_inicio AS inicio_1, d1.hora_fin AS fin_1,  
    d2.hora_inicio AS inicio_2, d2.hora_fin AS fin_2  
FROM disponibilidades d1  
JOIN disponibilidades d2  
    ON d1.id_usuario = d2.id_usuario  
    AND d1.fecha = d2.fecha  
    AND d1.id_disponibilidad < d2.id_disponibilidad  
    AND d1.hora_inicio < d2.hora_fin  
    AND d2.hora_inicio < d1.hora_fin;
```

Módulo de Tareas Asociadas a Eventos

Modelo Conceptual



Modelo Lógico

TAREAS (id_tarea [PK], titulo, descripcion, prioridad, fecha_limite, estado, id_evento [FK], id_usuario_responsable [FK])

Modelo Físico

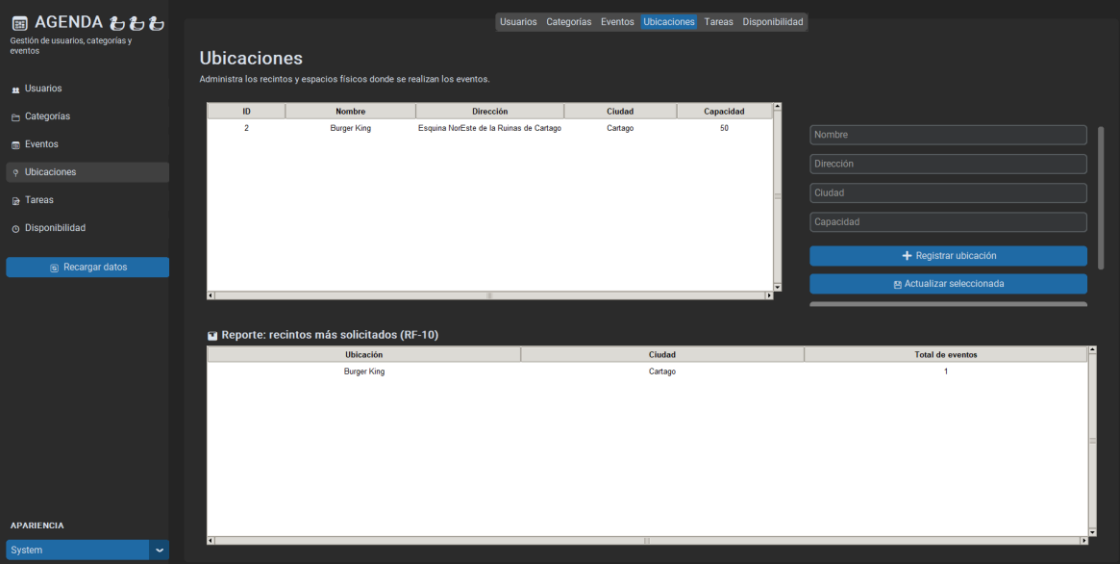
```
CREATE TABLE tareas (
    id_tarea SERIAL PRIMARY KEY,
    titulo VARCHAR(100) NOT NULL,
    descripcion VARCHAR(255),
    prioridad VARCHAR(20) NOT NULL,
    fecha_limite DATE NOT NULL,
    estado VARCHAR(20) NOT NULL DEFAULT 'Pendiente'
    CHECK (estado IN ('Pendiente', 'En progreso', 'Completada', 'Cancelada')),
    id_evento INT NOT NULL REFERENCES eventos(id_evento),
    id_usuario_responsable INT NOT NULL REFERENCES usuarios(id_usuario)
);

CREATE VIEW vista_carga_trabajo_usuarios AS
SELECT
    u.id_usuario, u.nombre, u.apellido,
    COUNT(*) FILTER (WHERE t.estado IN ('Pendiente', 'En progreso')) AS tareas_pendientes,
    COUNT(*) FILTER (WHERE t.estado IN ('Pendiente', 'En progreso') AND t.fecha_limite < CURRENT_DATE)
AS tareas_vencidas
FROM usuarios u
JOIN tareas t ON t.id_usuario_responsable = u.id_usuario
GROUP BY u.id_usuario, u.nombre, u.apellido;

CREATE VIEW vista_reporte_tareas_activas AS
SELECT
    t.id_tarea, u.nombre, u.apellido, t.titulo, t.estado, t.fecha_limite,
    (t.fecha_limite < CURRENT_DATE) AS esta_vencida
FROM tareas t
JOIN usuarios u ON u.id_usuario = t.id_usuario_responsable
WHERE t.estado IN ('Pendiente', 'En progreso');
```

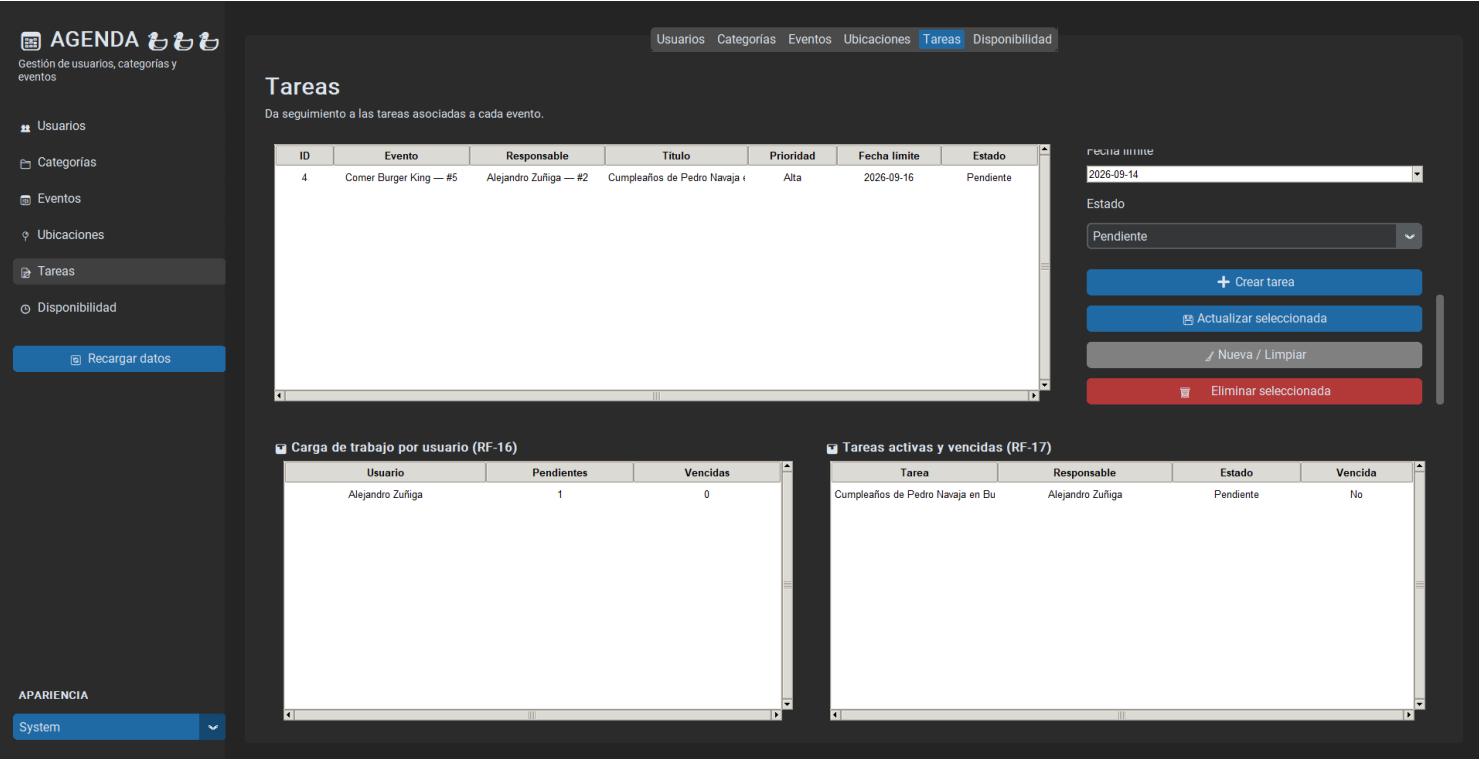
Evidencias Visuales

Pestaña de Ubicaciones



La tabla principal lista las ubicaciones registradas (nombre, dirección, ciudad y capacidad), con su formulario de registro, actualización y eliminación en el panel lateral derecho. Debajo se incorporó el panel de solo lectura (Reporte: recintos más solicitados (RF-10)), que muestra el ranking de ubicaciones ordenado por la cantidad de eventos que se han programado en cada una.

Pestaña de Tareas



La tabla principal muestra las tareas registradas junto con su evento, responsable, prioridad, fecha límite y estado, con su formulario de CRUD a la derecha. En la parte inferior se agregaron dos paneles de solo lectura: (Carga de trabajo por usuario (RF-16)), que indica cuántas tareas pendientes y vencidas tiene cada usuario, y (Tareas activas y vencidas (RF-17)), que lista las tareas activas señalando si ya vencieron su plazo.

Pestaña de Disponibilidad

AGENDA

Gestión de usuarios, categorías y eventos

Usuarios

Categorías

Eventos

Ubicaciones

Tareas

Disponibilidad

Recargar datos

APARIENCIA

System

UsuariosCategoríasEventosUbicacionesTareasDisponibilidad

Disponibilidad

Administra las franjas de disponibilidad de cada usuario.

ID	Usuario	Tipo	Fecha	Hora inicio	Hora fin
2	Alejandro Zuñiga — #2	Ocupado	2026-09-16	14:00:00	15:00:00

Traslapes de horario detectados (RF-12)

Usuario	Fecha	Franja 1	Franja 2
---------	-------	----------	----------

09:00

Hora fin (HH:MM)

10:00

Registrar franja

Actualizar seleccionada

Nueva / Limpiar

Eliminar seleccionada

La tabla principal lista las franjas de disponibilidad de cada usuario (tipo, fecha, hora de inicio y hora de fin), con su formulario de registro a la derecha. Debajo se incorporó el panel "Traslapes de horario detectados (RF-12)", que muestra automáticamente los cruces de horario entre franjas del mismo usuario en la misma fecha.

Pestaña de Eventos con el selector de Ubicación

AGENDA

Gestión de usuarios, categorías y eventos

Usuarios

Categorías

Eventos

Ubicaciones

Tareas

Disponibilidad

Recargar datos

APARIENCIA

System

UsuariosCategoríasEventosUbicacionesTareasDisponibilidad

Eventos

Programa eventos seleccionando usuarios, categorías, fechas y horas.

ID	Propietario	Categoría	Ubicación	Título	Inicio	Fin
5	Alejandro Zuñiga — #2	Chatarra — #3	Burger King — #2	Comer Burger King	2026-09-16 13:00	2026-09-16 14:00

Formulario de evento

Comer Burger King

Propietario

Alejandro Zuñiga — #2

Categoría

Chatarra — #3

Ubicación (opcional)

Burger King — #2

Inicio

2026-09-1613:00

Fin

2026-09-1614:00

Crear evento

Actualizar seleccionado

Nuevo / Limpiar

Eliminar seleccionado

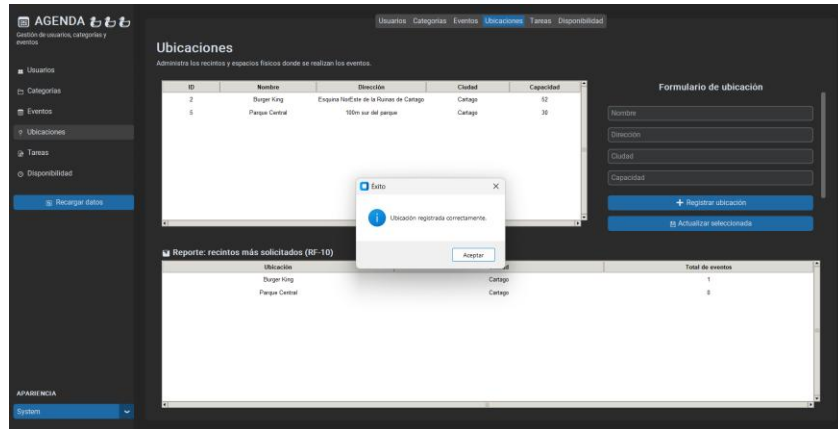
Se extendió el formulario original de Eventos agregando el campo "Ubicación (opcional)", que permite vincular cada evento con una de las ubicaciones registradas (RF-09). La tabla principal se actualizó para mostrar la columna Ubicación de cada evento, permitiendo consultar el histórico de eventos por recinto.

CRUD

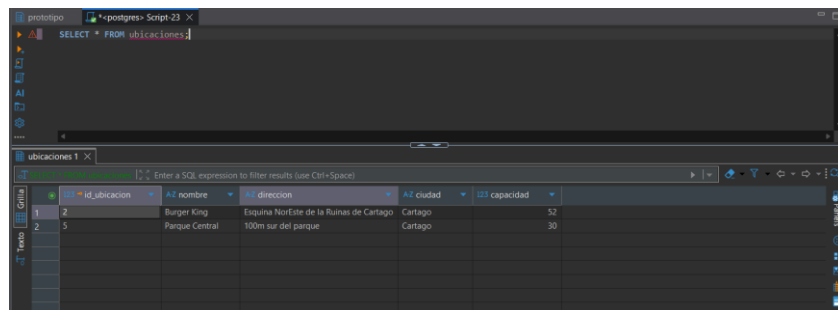
Para cada módulo se documenta el ciclo completo de Crear, Actualizar y Eliminar

CRUD de Modulo de Ubicaciones

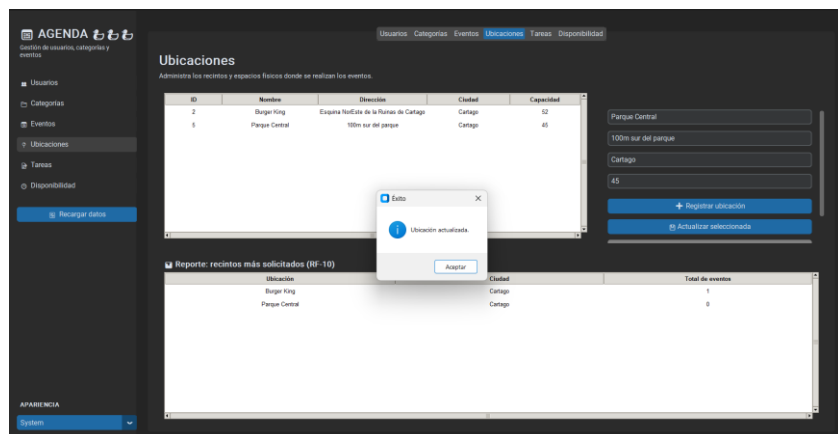
Crear: Se registra la ubicación "Parque Central" desde el formulario, el sistema confirma el registro exitoso.



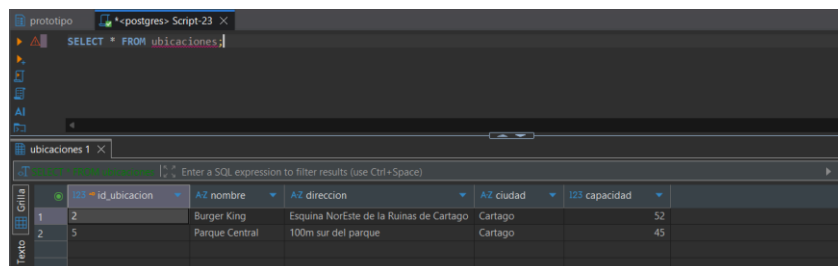
La consulta `SELECT * FROM ubicaciones;` confirma que la fila se guardó en PostgreSQL con capacidad 30.



Actualizar: Se selecciona "Parque Central" y se modifica su capacidad a 45; el sistema confirma la actualización.

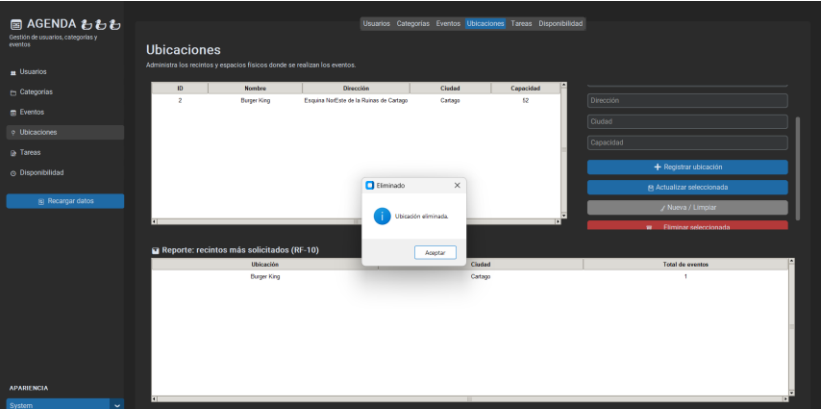


La consulta vuelve a ejecutarse y confirma que la capacidad ya quedó en 45 en la base de datos.

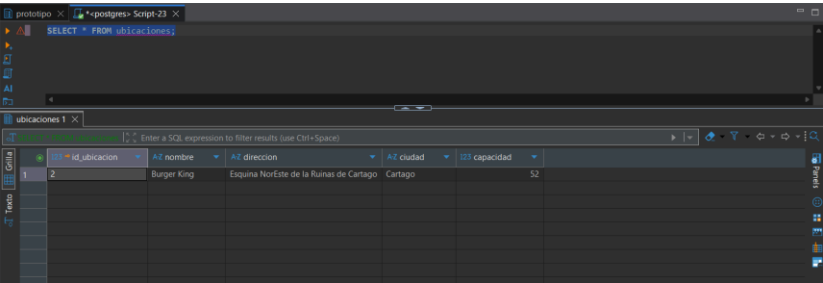


Eliminar: Se elimina la ubicación "Parque Central"; el sistema confirma la eliminación.

Se elimina la ubicación "Parque Central"; el sistema confirma la eliminación.

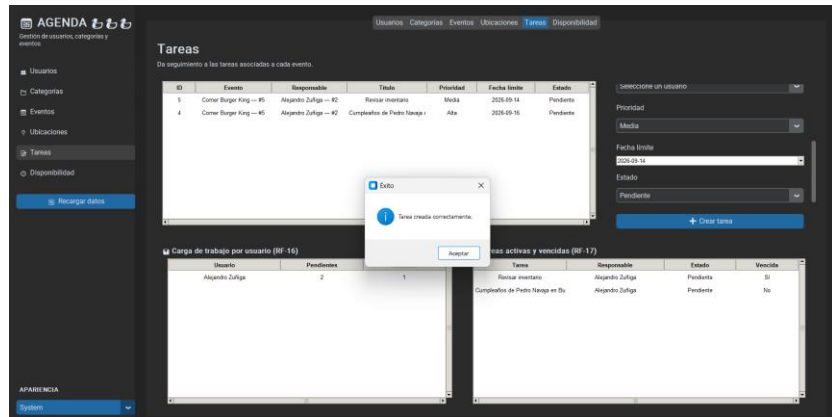


La consulta confirma que la fila ya no existe en la tabla ubicaciones.

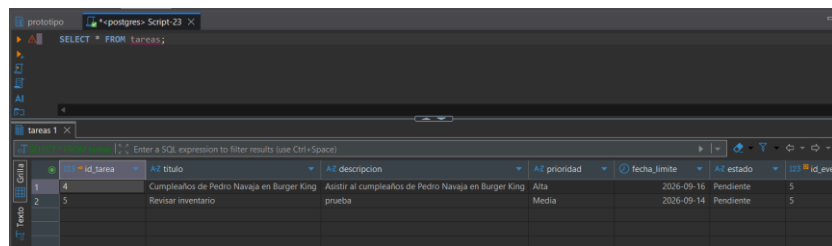


CRUD de Módulo de Tareas

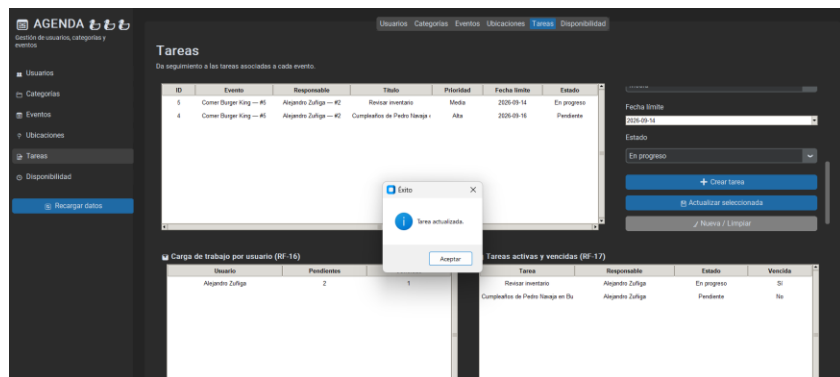
Crear: Se registra la tarea "Revisar inventario" asociada a un evento y un responsable; el sistema confirma el registro.



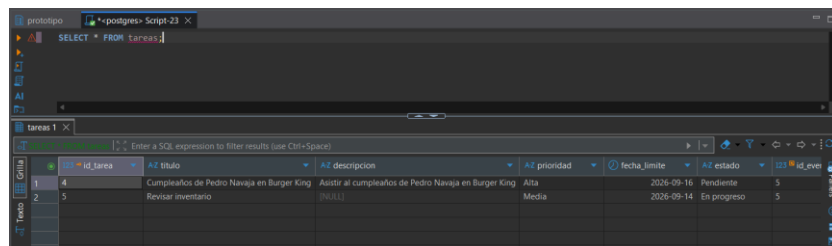
La consulta `SELECT * FROM tareas;` confirma que la tarea se guardó en PostgreSQL.



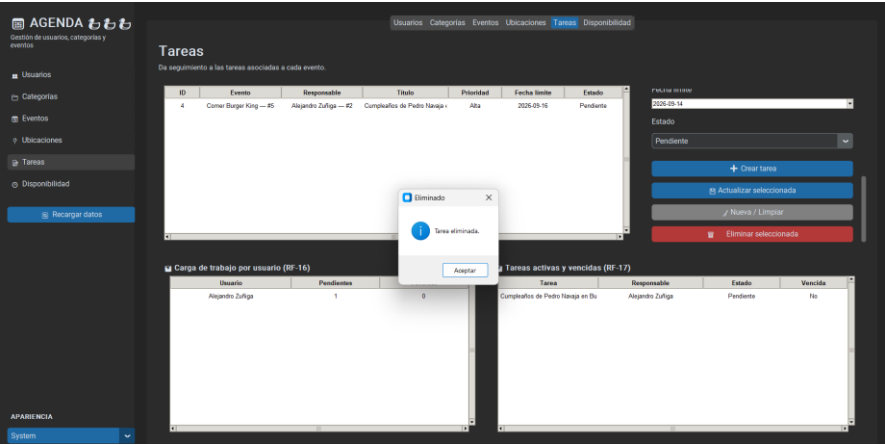
Actualizar: Se cambia el estado de la tarea a "En progreso", el sistema confirma la actualización.



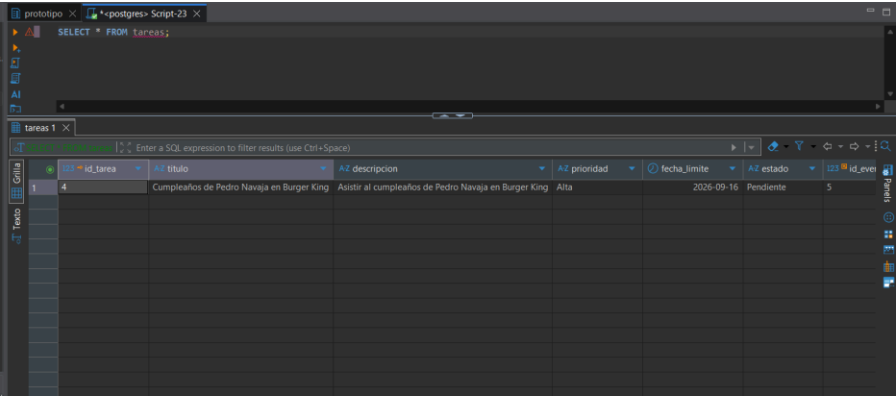
La consulta confirma que el estado quedó actualizado a "En progreso" en la base de datos.



Eliminar: Se elimina la tarea "Revisar inventario"; el sistema confirma la eliminación.

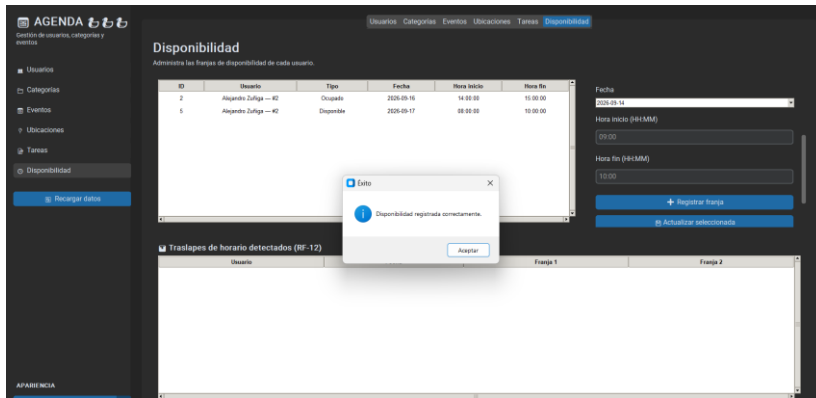


La consulta confirma que la tarea ya no existe en la tabla tareas.

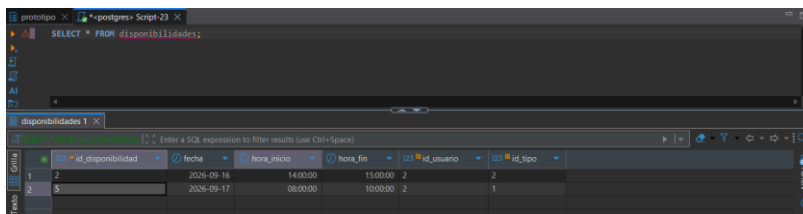


CRUD de Módulo de Disponibilidad

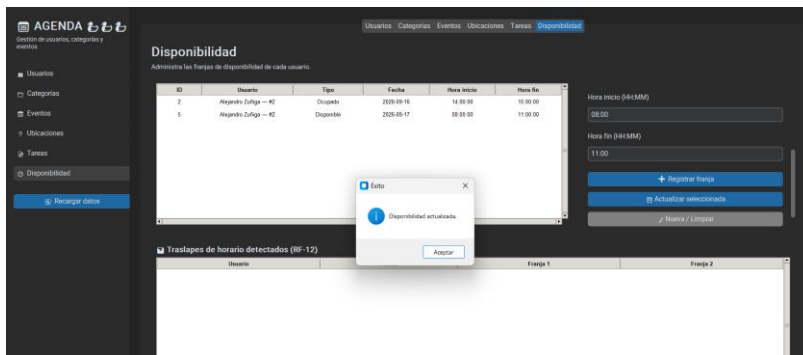
Crear: Se registra una franja de disponibilidad (08:00 a 10:00) para un usuario; el sistema confirma el registro.



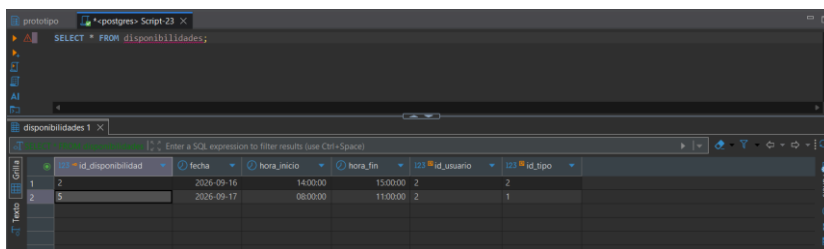
La consulta `SELECT * FROM disponibilidades;` confirma que la franja se guardó en PostgreSQL.



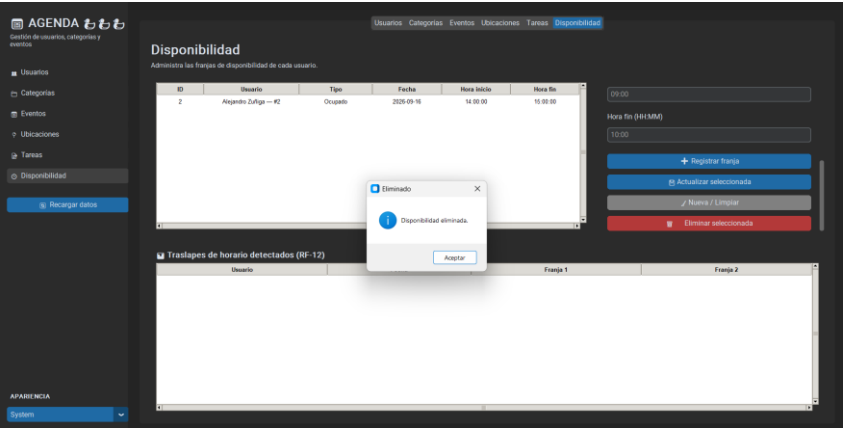
Actualizar: Se modifica la hora de fin de la franja a 11:00; el sistema confirma la actualización.



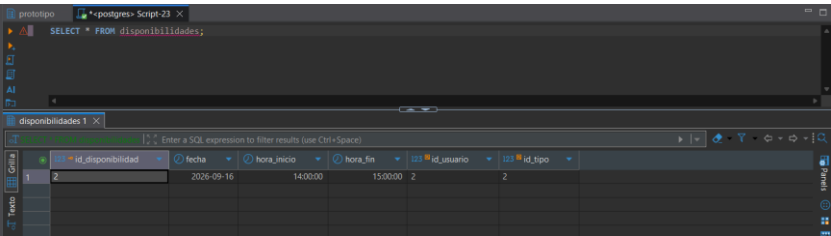
La consulta confirma que la hora de fin quedó actualizada a 11:00:00 en la base de datos.



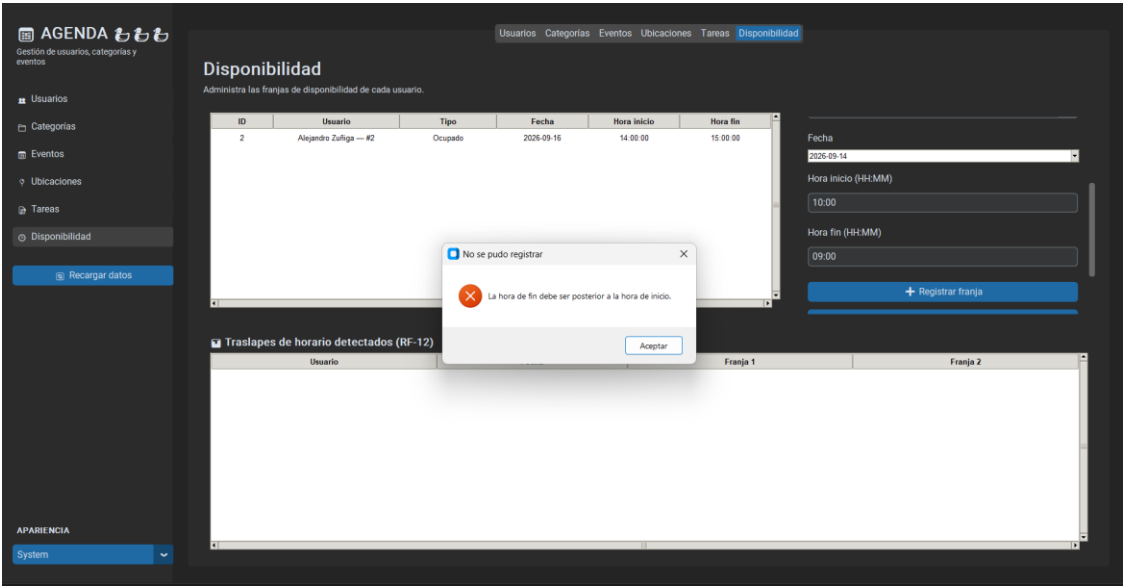
Eliminar: Se elimina la franja de disponibilidad registrada; el sistema confirma la eliminación.



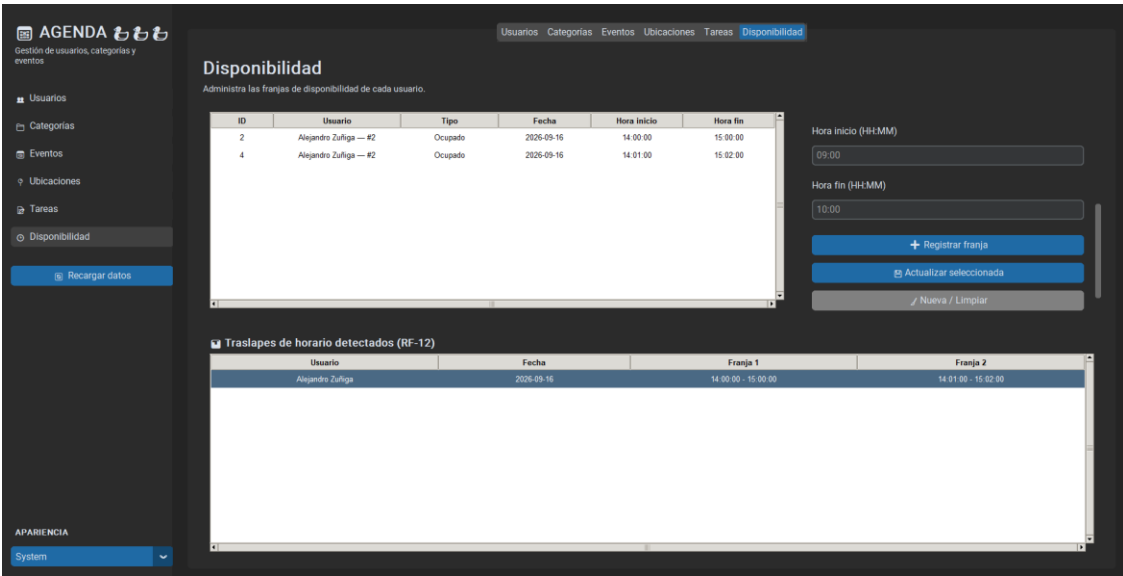
La consulta confirma que la franja ya no existe en la tabla disponibilidades.



Validación de Datos



Al intentar registrar una franja de disponibilidad con una hora de fin anterior a la hora de inicio, el sistema rechaza la operación y muestra un mensaje de error.



Al intentar registrar una franja de disponibilidad en un horario que ya está ocupado, el sistema no rechaza la operación pero identifica y registra un traslape en la tabla de inferior horarios.

Repositorio en Git

El código fuente y la documentación del proyecto se encuentran alojados de forma publica en el siguiente repositorio: <https://github.com/azuniga7/Proyecto1-Agenda-tres-Patitos>